

Obyektga yo'naltirilgan dasturlash

Klasslar va obyektlar

Python'da **int**, **str** kabi ko'plab ichki (standart) o'zgaruvchilar turlari mavjud bo'lib, biz ularni dasturda ishlata olamiz. Lekin Python o'zimizning turlarimizni ham **klasslar** yordamida aniqlash imkonini beradi. **Klass** muayyan bir mohiyatni ifodalaydi. Klassning aniq ko'rinishi esa **obyekt** hisoblanadi.

Buni quyidagi o'xshatish orqali ham tushuntirish mumkin. Bizda inson haqida bir tushuncha bor: insonning ismi, yoshi va boshqa xususiyatlari bo'ladi. Inson ayrim harakatlarni bajarishi mumkin – yurish, yugurish, o'ylash va hokazo. Demak, inson haqida umumiy tushuncha – bu klassdir. Ammo bu andozaning aniq ko'rinishi farqlanadi: kimdadir bir ism, boshqasida boshqa ism bo'ladi. Haqiqiy mavjud inson esa ushbu klassning **obyekti** hisoblanadi.

Python tilida klass **class** kalit so'zi yordamida aniqlanadi:

```
class klass_nomi:
    klass_atributlari
    klass_metodlari
```

Klass ichida uning **atributlari** aniqlanadi, ular klassning turli xususiyatlarini saqlaydi, shuningdek, **metodlari** – ya'ni klass funksiyalari belgilanadi.

Keling, eng oddiy klassni yaratamiz:

```
class Person:
    pass
```

Bu misolda **Person** nomli klass aniqlangan bo'lib, u shartli ravishda insonni ifodalaydi. Ushbu klassda hozircha hech qanday metod yoki atribut belgilanmagan. Biroq klass ichida hech narsa yozilmasa bo'lmaydi, shuning uchun **pass** operatoridan foydalaniladi. Bu operator kod yozilishi kerak bo'lgan joyda ishlatiladi, lekin vazifaga ko'ra bizga kod kerak bo'lmaganda shunchaki bo'sh o'rin to'ldiruvchi sifatida qo'yiladi.

Klass yaratilgandan keyin uning obyektlarini aniqlash mumkin. Masalan:

```
class Person:
    pass

tom = Person()      # tom obyektini aniqlash
bob = Person()      # bob obyektini aniqlash
```

Person klassi aniqlangandan so'ng undan ikkita obyekt – **tom** va **bob** yaratilmoqda. Obyekt yaratishda maxsus funksiya – **konstruktor** qo'llaniladi. Konstruktor klass nomi bilan chaqiriladi va u klass obyektini qaytaradi. Ya'ni, bu misolda **Person()** chaqirig'i konstruktorga murojaat qilishni bildiradi.

Har bir klass odatda sukut bo'yicha **parametrlarsiz konstruktorga** ega bo'ladi:

```
tom = Person()      # Person() - konstruktorga murojaat, u Person klassining
obyektini qaytaradi
```

Konstruktorlar

Demak, obyekt yaratishda **konstruktor** ishlatiladi. Yuqoridagi misolda biz **Person** klassidan obyektlar yaratishda sukut bo'yicha mavjud bo'lgan, **parametrlarsiz konstruktordan** foydalandik. Bunday konstruktor barcha klasslarda yashirin holda mavjud bo'ladi.

Lekin biz o'zimiz ham konstruktorni aniq belgilashimiz mumkin. Buning uchun maxsus metod – `__init__()` (ikkita pastki chiziq bilan) ishlatiladi.

Masalan, **Person** klassiga konstruktor qo'shamiz:

```
class Person:
    # konstruktor
    def __init__(self):
        print("Person obyektini yaratish")

tom = Person()      # Person obyektini yaratish
```

Bu yerda **Person** klassida konstruktor – `__init__` funksiyasi aniqlangan. Konstruktor **kamida bitta parametr**, ya'ni joriy obyektga ishora qiluvchi **self** parametrini qabul qilishi kerak.

Konstruktor odatda obyekt yaratilishida bajarilishi kerak bo'lgan ishlarni belgilash uchun ishlatiladi.

Endi obyekt yaratilganda:

```
tom = Person()
```

konstruktor `__init__()` chaqiriladi va ekranga "Person obyektini yaratish" satri chiqariladi.

🔴 Muhim eslatma: Konstruktor aslida oddiy funksiya, faqat uni chaqirish uchun **klass nomi** ishlatiladi, `__init__` bevosita chaqirilmaydi. Shuningdek, konstruktor chaqirilganda **self** parametriga qiymatni biz uzatmaymiz – Python uni avtomatik tarzda obyektning o'ziga bog'lab beradi.

Obyekt atributlari

Atributlar obyektning holatini saqlaydi. Klass ichida atributlarni aniqlash va ularga qiymat berish uchun **self** kalit so'zi ishlatiladi.

Masalan, quyidagi **Person** klassini ko'raylik:

```
class Person:
    def __init__(self, name, age):
        self.name = name      # insonning ismi
        self.age = age        # insonning yoshi

tom = Person("Tom", 22)

# atributlarga murojaat qilish
# qiymat olish
```

```
print(tom.name)      # Tom
print(tom.age)       # 22

# qiymatni o'zgartirish
tom.age = 37
print(tom.age)       # 37
```

 Bu yerda konstruktor ikki parametr – **name** va **age** qabul qiladi. Konstruktor ichida ular obyekt atributlariga (`self.name`, `self.age`) biriktiriladi.

Agar biz konstruktorni parametrlar bilan e'lon qilsak (`self` dan tashqari), konstruktor chaqirilganda ularga qiymat berishimiz shart:

```
tom = Person("Tom", 22)  # name="Tom", age=22
```

Endi obyekt nomi orqali atributlarga murojaat qilib, ularning qiymatini olish yoki o'zgartirish mumkin:

```
print(tom.name)  # "Tom"
tom.age = 37     # age atributini yangilash
```

Turli atribut qiymatlari bilan obyektlar yaratish

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

tom = Person("Tom", 22)
bob = Person("Bob", 43)

print(tom.name)  # Tom
print(bob.name)  # Bob
```

 Yuqorida ikkita obyekt – **tom** va **bob** yaratildi. Ularning atributlar to'plami bir xil, lekin qiymatlari har xil. Har safar obyekt yaratilganda Python avtomatik ravishda `self` orqali obyektни bog'laydi.

- `tom = Person("Tom", 22)` → obyekt **tom** bo'ladi.
- `bob = Person("Bob", 43)` → obyekt **bob** bo'ladi.

Dinamik atribut qo'shish

Atributlarni faqat konstruktor ichida emas, balki keyin ham qo'shish mumkin:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

tom = Person("Tom", 22)
```

```
# yangi atribut qo'shish
tom.company = "Microsoft"
print(tom.company) # Microsoft
```

Bu yerda **company** atributi obyekt yaratilgandan keyin dinamik ravishda qo'shildi.

⚠ Biroq bunday usul xatolarga olib kelishi mumkin. Masalan, agar atributni e'lon qilmasdan oldin unga murojaat qilsak:

```
tom = Person("Tom", 22)
print(tom.company) # ✗ AttributeError: Person obyektida company atributi yo'q
```

👉 Shu sababli atributlarni odatda konstruktor ichida aniqlash tavsiya etiladi.

Klass metodlari

Klass metodlari – bu aslida klass ichida aniqlangan funksiyalar bo'lib, ular obyektning xatti-harakatlarini belgilaydi.

Masalan, oddiy **Person** klassini olaylik:

```
class Person: # Person klassi ta'rifi
    def say_hello(self):
        print("Hello")

tom = Person()
tom.say_hello() # Hello
```

Bu yerda **say_hello()** metodi yozilgan. U shunchaki salomlashish uchun "Hello" matnini chiqaradi.

🔴 Muhim jihat: konstruktor (`__init__`) singari har bir metodning ham birinchi parametri **self** bo'lishi kerak. **self** – bu joriy obyektga ishora qiladi. Biroq metodni chaqirganda **self** parametriga hech qanday qiymat berilmaydi – Python uni avtomatik tarzda uzatadi.

Metodlarni chaqirish

Metodlarga obyekt nomi orqali nuqta (.) bilan murojaat qilinadi:

```
obyekt.metod([parametrlar])
```

Masalan:

```
tom.say_hello() # Hello
```

Parametrli metod

Metod qo'shimcha parametrlar ham qabul qilishi mumkin:

```
class Person:
    def say(self, message):      # metod
        print(message)

tom = Person()
tom.say("Hello Python")        # Hello Python
```

Bu yerda metod **say()** ikkita parametrga ega:

- **self** – obyektning o‘zi
- **message** – tashqaridan beriladigan xabar

Metodni chaqirganda **message** qiymatini berishimiz kerak bo‘ladi.

Metod ichida atributlarga murojaat qilish

Obyektning atributlari va boshqa metodlariga klass ichida **self** orqali murojaat qilinadi:

```
self.atribut      # atributga murojaat
self.metod()      # metodga murojaat
```

Misol: obyekt haqida ma’lumot chiqaruvchi metod

```
class Person:
    def __init__(self, name, age):
        self.name = name      # insonning ismi
        self.age = age        # insonning yoshi

    def display_info(self):    # obyekt haqida ma’lumot chiqarish
        print(f"Name: {self.name} Age: {self.age}")

tom = Person("Tom", 22)
tom.display_info()           # Name: Tom Age: 22

bob = Person("Bob", 43)
bob.display_info()           # Name: Bob Age: 43
```

🔗 Bu yerda `display_info()` metodi obyekt atributlariga (`self.name`, `self.age`) murojaat qilib, ularni konsolga chiqaradi.

Konsoldagi natija:

```
Name: Tom Age: 22
Name: Bob Age: 43
```

Destruktorlar

Klasslarda konstruktorlardan tashqari **destruktorlar** ham mavjud. Destruktor — bu obyekt o‘chirib tashlanganda avtomatik chaqiriladigan maxsus metoddir.

Python'da destruktör `__del__(self)` shaklida yoziladi. U ham konstruktor kabi **self** parametrini qabul qiladi va odatda obyekt foydalanayotgan resurslarni bo'shatish yoki o'chirish uchun ishlatiladi.

🔗 Muhimi: destruktorni biz o'zimiz chaqirmaymiz, uni Python interpretatori avtomatik chaqiradi.

Oddiy misol

```
class Person:
    def __init__(self, name):
        self.name = name
        print("Ismi", self.name, "bo'lgan odam yaratildi")

    def __del__(self):
        print("Ismi", self.name, "bo'lgan odam o'chirildi")

tom = Person("Tom")
```

👉 Bu dasturda:

- Obyekt yaratilganda konstruktor ishlaydi va "Ismi Tom bo'lgan odam yaratildi" degan yozuv chiqadi.
- Dastur tugaganda yoki obyekt yo'q qilinganda destruktör chaqiriladi va "Ismi Tom bo'lgan odam o'chirildi" chiqadi.

Konsoldagi natija:

```
Ismi Tom bo'lgan odam yaratildi
Ismi Tom bo'lgan odam o'chirildi
```

Funksiya ichida obyekt yaratish

```
class Person:
    def __init__(self, name):
        self.name = name
        print("Ismi", self.name, "bo'lgan odam yaratildi")

    def __del__(self):
        print("Ismi", self.name, "bo'lgan odam o'chirildi")

def create_person():
    tom = Person("Tom")    # obyekt faqat shu funksiya ichida yashaydi

create_person()
print("Dastur tugadi")
```

👉 Bu yerda obyekt **create_person()** funksiyasi ichida yaratiladi. Funksiya ishlashini tugatganda obyektning hayoti ham tugaydi va destruktör avtomatik ravishda chaqiriladi.

Konsoldagi natija:

```
Ismi Tom bo'lgan odam yaratildi  
Ismi Tom bo'lgan odam o'chirildi  
Dastur tugadi
```

Inkapsulyatsiya, atributlar va xususiyatlar

Odatiy holatda klasslardagi atributlar **umumiy ochiq** hisoblanadi, ya'ni dasturning istalgan joyidan obyektning atributini olishimiz va uni o'zgartirishimiz mumkin. Masalan:

```
class Person:
    def __init__(self, name, age):
        self.name = name    # ismni o'rnatamiz
        self.age = age      # yoshni o'rnatamiz

    def print_person(self):
        print(f"Ism: {self.name}\tYosh: {self.age}")

tom = Person("Tom", 39)
tom.name = "Odam-o'rgimchak"    # name atributini o'zgartiramiz
tom.age = -129                  # age atributini o'zgartiramiz
tom.print_person()              # Ism: Odam-o'rgimchak    Yosh: -129
```

Ammo bu holda biz, masalan, insonning yoshiga yoki ismining qiymatiga **noto'g'ri ma'lumot** berishimiz mumkin, masalan, manfiy yosh ko'rsatish. Bunday xatti-harakatlar **nojoiz** hisoblanadi. Shuning uchun atributlarga kirishni nazorat qilish masalasi paydo bo'ladi.

Ushbu muammo **inkapsulyatsiya** tushunchasi bilan chambarchas bog'liq. Inkapsulyatsiya obyektga tashqaridan bevosita murojaat qilishni cheklash va funktsionallikni yashirishni nazarda tutuvchi obyektga yo'naltirilgan dasturlashning asosiy konsepsiyalaridan biridir.

Python dasturlash tili **privat yoki yopiq atributlarni** belgilash imkonini beradi. Buning uchun atribut nomi ikki pastki chiziq bilan boshlanishi kerak – `__name`.

Masalan, oldingi dasturimizni `name` va `age` atributlarini **privat** qilib yozamiz:

```
class Person:
    def __init__(self, name, age):
        self.__name = name    # ismni o'rnatamiz
        self.__age = age      # yoshni o'rnatamiz

    def print_person(self):
        print(f"Ism: {self.__name}\tYosh: {self.__age}")

tom = Person("Tom", 39)
tom.__name = "Odam-o'rgimchak"    # __name atributini o'zgartirishga urinamiz
tom.__age = -129                  # __age atributini o'zgartirishga urinamiz
tom.print_person()              # Ism: Tom    Yosh: 39
```

Ko'rinib turibdiki, biz `__name` va `__age` atributlariga **yangi qiymat berishga urinsak ham**, ular o'zgarib ketmaydi. Chunki ular **privat (yopiq)** hisoblanadi va tashqaridan bevosita o'zgartirib bo'lmaydi.

```
tom.__name = "Odam-o'rgimchak"    # __name atributini o'zgartirishga urinish
```



```
tom.__age = -129 # __age atributini o'zgartirishga urinish
```

Bu urinishlar aslida **obyekt ichidagi haqiqiy atributlarga ta'sir qilmaydi**, balki yangi tashqi atributlar yaratib qo'yadi.

Lekin `print_person` metodi chaqirilganda ko'ramizki, atributlarning qiymati o'zgarmagan bo'ladi:

```
tom.print_person() # Ism: Tom Yosh: 39
```

Bu qanday ishlaydi?

Agar atribut nomi ikki pastki chiziq bilan boshlangan bo'lsa, masalan `__attribute`, Python aslida uni maxsus shaklda saqlaydi — `__ClassName__attribute` ko'rinishida.

Ya'ni yuqoridagi misolda atributlar:

- `__name` → `__Person__name`
- `__age` → `__Person__age`

shaklida yaratiladi.

Shuning uchun ularga faqat shu klass ichidan murojaat qilish mumkin, tashqaridan emas. Masalan, quyidagicha qiymat berish hech qanday natija bermaydi:

```
tom.__age = 43
```

Sababi bu holda Python obyektga shunchaki **yangi atribut** (`__age`) yaratadi, lekin u mavjud bo'lgan `self.__age` (aniqrog'i `self.__Person__age`) bilan bog'liq emas.

👉 Shu sababli `print_person()` metodida hanuz eski qiymat — 39 chiqaveradi.

Agar `__age` atributi ilgari aniqlanmagan bo'lsa, uning qiymatini olishga urinish xatolikka olib keladi:

```
print(tom.__age)
```

Shunga qaramay, bu yerda atributlarning **privatligi nisbiy** hisoblanadi. Chunki biz to'liq nomdan foydalanib ularga murojaat qilishimiz mumkin.

Masalan:

```
class Person:
    def __init__(self, name, age):
        self.__name = name # ismni o'rnatamiz
        self.__age = age # yoshni o'rnatamiz

    def print_person(self):
        print(f"Ism: {self.__name}\tYosh: {self.__age}")

tom = Person("Tom", 39)
```

```
tom.__Person__name = "Odam-o'rgimchak" # __name atributini o'zgartiramiz
tom.print_person() # Ism: Odam-o'rgimchak Yosh: 39
```

Ya'ni, texnik jihatdan atributni o'zgartirish mumkin, lekin **klass tashqarisidagi dasturchi** atributning aniq nomini (`__Person__name`) bilishi yoki taxmin qilishi kerak bo'ladi. Shu sababli bu mexanizm **haqiqiy himoya emas, balki intizom va kelishuvga asoslangan yopish** usuli hisoblanadi.

Ruxsat metodlari. Getter va Setterlar

Shunday savol tug'ilishi mumkin: **privat atributlarga qanday murojaat qilish kerak?** Buning uchun maxsus **kirish metodlari** ishlatiladi:

- **Getter** → atribut qiymatini olish uchun xizmat qiladi.
- **Setter** → atribut qiymatini o'rnatish (o'zgartirish) uchun xizmat qiladi.

Endi yuqorida ko'rib chiqqan klassimizni getter va setter metodlari bilan to'ldiramiz:

```
class Person:
    def __init__(self, name, age):
        self.__name = name # ismni o'rnatamiz
        self.__age = age # yoshni o'rnatamiz

    # setter – yoshni o'rnatish
    def set_age(self, age):
        if 0 < age < 110:
            self.__age = age
        else:
            print("Yosh noto'g'ri kiritildi")

    # getter – yoshni olish
    def get_age(self):
        return self.__age

    # getter – ismni olish
    def get_name(self):
        return self.__name

    def print_person(self):
        print(f"Ism: {self.__name}\tYosh: {self.__age}")

tom = Person("Tom", 39)
tom.print_person() # Ism: Tom Yosh: 39
tom.set_age(-3486) # Yosh noto'g'ri kiritildi
tom.set_age(25)
tom.print_person() # Ism: Tom Yosh: 25
```

👉 Bu usul yordamida atributlarni to'g'ridan-to'g'ri o'zgartirish mumkin bo'lmaydi, faqat **setter orqali shart bilan** yangilash mumkin. Shu orqali **ma'lumotni himoya qilish va nazorat ostida ushlab turish** ta'minlanadi.

Yosh qiymatini olish uchun `get_age` metodi qo'llaniladi:

```
def get_age(self):  
    return self.__age
```

Yoshni o'zgartirish uchun esa `set_age` metodi aniqlangan:

```
def set_age(self, age):  
    if 0 < age < 110:  
        self.__age = age  
    else:  
        print("Yosh noto'g'ri kiritildi")
```

Atributlarga **metodlar orqali bilvosita murojaat qilish** qo'shimcha mantiq qo'shish imkonini beradi. Masalan, berilgan yosh qiymatiga qarab biz uni qayta o'rnatish kerakmi yoki yo'qligini hal qilamiz, chunki kiritilgan qiymat noto'g'ri bo'lishi mumkin.

Shuningdek, har bir yopiq atribut uchun alohida **getter-setter juftligi** yaratish shart emas. Masalan, yuqoridagi misolda insonning ismi faqat konstruktor orqali o'rnatiladi, uni tashqaridan o'zgartirib bo'lmaydi. Faqatgina ismni olish uchun `get_name` metodi belgilangan.

Xususiyatlar annotatsiyasi

Yuqorida biz **getter** va **setter** metodlarini qanday yaratishni ko'rdik. Ammo Python buni yanada **elegant usulda** bajarishga imkon beradi — **xususiyatlar (properties)** orqali.

Bunda maxsus **annotatsiyalar** ishlatiladi (@ belgisi bilan boshlanadi):

- `@property` → xususiyat-getterni yaratadi.
- `<getter nomi>.setter` → shu getterga mos keluvchi xususiyat-setterni yaratadi.

Endi `Person` klassini xususiyat annotatsiyalari yordamida yozamiz:

```
class Person:  
    def __init__(self, name, age):  
        self.__name = name    # ismni o'rnatamiz  
        self.__age = age      # yoshni o'rnatamiz  
  
    # xususiyat-getter  
    @property  
    def age(self):  
        return self.__age  
  
    # xususiyat-setter  
    @age.setter  
    def age(self, age):  
        if 0 < age < 110:  
            self.__age = age  
        else:  
            print("Yosh noto'g'ri kiritildi")
```

```

@property
def name(self):
    return self.__name

def print_person(self):
    print(f"Ism: {self.__name}\tYosh: {self.__age}")

tom = Person("Tom", 39)
tom.print_person()    # Ism: Tom  Yosh: 39
tom.age = -3486        # Yosh noto'g'ri kiritildi (setter ishladi)
print(tom.age)         # 39 (getter ishladi)
tom.age = 25           # setter ishladi
tom.print_person()    # Ism: Tom  Yosh: 25

```

Birinchiidan, shuni ta'kidlash kerakki, **xususiyat-setter** faqat **xususiyat-getter**dan keyin aniqlanadi.

Ikkinchiidan, **getter** ham, **setter** ham bir xil nomga ega bo'ladi — bizning misolimizda **age**. Getter **age** deb nomlangani sababli, setter ustida ham **@age.setter** annotatsiyasi qo'llanadi.

Shundan keyin ham getterga, ham setterga biz bir xil tarzda murojaat qilamiz:

```
tom.age
```

👉 Bunda Python ichida kerakli metod (getter yoki setter) avtomatik tanlanadi:

- `print(tom.age)` chaqirilsa → getter ishlaydi
- `tom.age = 25` yozilsa → setter ishlaydi

Shuningdek, **faqat getter**ni ham belgilash mumkin. Masalan, biz `name` xususiyati uchun **faqat getter** yozdik, shuning uchun uni tashqaridan o'zgartirib bo'lmaydi, faqat o'qish mumkin.

```

print(tom.name)    # ismni olish mumkin
tom.name = "Ali"   # xato — o'zgartirib bo'lmaydi

```

Vorislik

Vorislik mavjud bo'lgan klass asosida yangi klass yaratish imkonini beradi. Inkapsulyatsiya bilan bir qatorda vorislik obyektga yo'naltirilgan dasturlashning asosiy tayanch tushunchalaridan biridir.

Vorislikning asosiy tushunchalari — **subklass** va **superklass**. Subklass superklassdan barcha ommaviy (public) atributlar va metodlarni voris sifatida oladi. Superklass shuningdek **bazaviy klass (base class)** yoki **ota klass (parent class)** deb ham ataladi, subklass esa **hosila klass (derived class)** yoki **farzand klass (child class)** deb yuritiladi.

Klasslar vorisligi uchun sintaksis quyidagicha ko'rinadi:

```
class Subklass(Superklass):  
    subclass_metodlari
```

Masalan, bizda **Person** klassi mavjud bo'lib, u odamni ifodalaydi:

```
class Person:  
  
    def __init__(self, name):  
        self.__name = name    # odamning ismi  
  
    @property  
    def name(self):  
        return self.__name  
  
    def display_info(self):  
        print(f"Name: {self.__name} ")
```

Faraz qilaylik, bizga korxonada ishlovchi **xodim klassi** kerak bo'ldi. Buning uchun biz yangidan yangi klass yaratishimiz mumkin, masalan, **Employee** klassi:

```
class Employee:  
  
    def __init__(self, name):  
        self.__name = name    # xodimning ismi  
  
    @property  
    def name(self):  
        return self.__name  
  
    def display_info(self):  
        print(f"Name: {self.__name} ")  
  
    def work(self):  
        print(f"{self.name} ishlayotganda")
```

Biroq **Employee** klassi ham **Person** klassidagi atribut va metodlarga ega bo'lishi mumkin, chunki xodim — bu odam. Yuqoridagi misolda **Employee** klassiga faqat `work` metodi qo'shilgan, qolgan kod esa **Person** klassining funktsionalligini takrorlaydi. Lekin bitta klass funktsionalligini boshqasida takrorlamaslik uchun bu yerda **vorislik**dan foydalanish maqsadga muvofiqdir.

Shunday qilib, **Employee** klassini **Person** klassidan voris qilamiz:

```
class Person:

    def __init__(self, name):
        self.__name = name    # odamning ismi

    @property
    def name(self):
        return self.__name

    def display_info(self):
        print(f"Name: {self.__name} ")

class Employee(Person):

    def work(self):
        print(f"{self.name} ishlamoqda")

tom = Employee("Tom")
print(tom.name)      # Tom
tom.display_info()   # Name: Tom
tom.work()           # Tom ishlamoqda
```

Employee klassi **Person** klassining barcha funktsionalligini oladi, faqat `work()` metodini qo'shadi. Shuning uchun **Employee** obyektini yaratishda **Person** konstruktoridan foydalanish mumkin:

```
tom = Employee("Tom")
```

Voris qilingan atributlar va metodlarga murojaat qilish ham mumkin:

```
print(tom.name)      # Tom
tom.display_info()   # Name: Tom
```

Lekin **Employee** yopiq atributlarga (`__name`) kira olmaydi. Masalan, quyida xato yuz beradi:

```
def work(self):
    print(f"{self.__name} works")    # ! Xato
```

Ko'p martalik vorislik

Python tilining o'ziga xos xususiyatlaridan biri bu **ko'p martalik vorislik**ni qo'llab-quvvatlashidir, ya'ni bitta klass bir nechta klasslardan voris olishi mumkin:

```
# xodim klassi
class Employee:
    def work(self):
        print("Employee works")

# talaba klassi
```

```

class Student:
    def study(self):
        print("Student studies")

class WorkingStudent(Employee, Student):  # Employee va Student klasslaridan
    vorislik
    pass

# ishlovchi talaba klassi
tom = WorkingStudent()
tom.work()      # Employee works
tom.study()     # Student studies

```

Bu yerda **Employee** klassi xodimni, **Student** klassi talabani ifodalaydi. **WorkingStudent** klassi esa ishlovchi talabani bildiradi. Unda yangi funksional belgilanmagan, shuning uchun `pass` operatori yozilgan. **WorkingStudent** klassi shunchaki **Employee** va **Student** klasslaridan funktsionallikni voris qiladi. Shu sababli obyekt orqali ikkala klass metodlarini ham chaqirish mumkin.

Vorislik qilinayotgan klasslar yanada murakkab funktsionallikka ega bo'lishi mumkin, masalan:

```

class Employee:

    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    def work(self):
        print(f"{self.name} works")

class Student:

    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    def study(self):
        print(f"{self.name} studies")

class WorkingStudent(Employee, Student):
    pass

```

```
tom = WorkingStudent("Tom")
tom.work()      # Tom works
tom.study()     # Tom studies
```

Ko‘p martalik vorislik qulay bo‘lib ko‘rinishi mumkin, ammo ikkala bazaviy klassda bir xil nomli metod/atribut bo‘lsa, chalkashlikka olib keladi. Masalan:

```
class Employee:
    def do(self):
        print("Employee works")

class Student:
    def do(self):
        print("Student studies")

# class WorkingStudent(Student, Employee):
class WorkingStudent(Employee, Student):
    pass

tom = WorkingStudent()
tom.do()      # ?
```

Bu yerda ikkala bazaviy klass — **Employee** va **Student** `do` metodini belgilagan, lekin ular turlicha natija chiqaradi. Savol: **WorkingStudent** klassi qaysi metodni ishlatadi?

Agar klass e‘lon qilinayotganda birinchi o‘rinda **Employee** turgan bo‘lsa:

```
class WorkingStudent(Employee, Student)
```

unda `do` metodi **Employee** klassidan olinadi.

Agar tartibni almashtirsak:

```
class WorkingStudent(Student, Employee)
```

unda `do` metodi **Student** klassidan olinadi.

Bazaviy klasslar funksionalligi qaysi tartibda qo‘llanishini dasturiy tarzda tekshirish mumkin. Buning uchun `__mro__` atributi yoki `mro()` metodidan foydalaniladi:

```
print(WorkingStudent.__mro__)
print(WorkingStudent.mro())
```


Bazaviy klass funksionalligini qayta aniqlash (override)

Oldingi maqolada **Employee** klassi **Person** klassining barcha funksionalligini olgan edi:

```
class Person:

    def __init__(self, name):
        self.__name = name    # odamning ismi

    @property
    def name(self):
        return self.__name

    def display_info(self):
        print(f"Name: {self.__name} ")

class Employee(Person):

    def work(self):
        print(f"{self.name} works")
```

Lekin agar biz bu funksionallikning ayrim qismlarini o'zgartirmoqchi bo'lsak-chi? Masalan, xodimga konstruktor orqali yangi atribut — uning ishlaydigan kompaniyasini saqlovchi **company** qo'shish yoki `display_info` metodini o'zgartirish. Python bazaviy klass funksionalligini qayta aniqlash (**override**) imkonini beradi.

Masalan, klasslarni quyidagicha o'zgartiramiz:

```
class Person:

    def __init__(self, name):
        self.__name = name    # odamning ismi

    @property
    def name(self):
        return self.__name

    def display_info(self):
        print(f"Name: {self.__name}")

class Employee(Person):

    def __init__(self, name, company):
        super().__init__(name)    # Person klassi konstruktorini chaqirish
        self.company = company

    def display_info(self):
        super().display_info()    # Person klassidagi display_info() chaqiriladi
        print(f"Company: {self.company}")
```

```
def work(self):
    print(f"{self.name} works")

tom = Employee("Tom", "Microsoft")
tom.display_info()
# Name: Tom
# Company: Microsoft
```

Bu yerda **Employee** klassiga yangi atribut — `self.company` qoʻshiladi, u xodimning kompaniyasini saqlaydi. Shunga muvofiq, `__init__()` metodi uchta parametr qabul qiladi: ikkinchisi ismni, uchinchi esa kompaniyani oʻrnatish uchun.

Agar bazaviy klassda konstruktor (`__init__`) aniqlangan boʻlsa va biz hosila klassda uni oʻzgartirmoqchi boʻlsak, unda hosila klass konstruktorida bazaviy klass konstruktorini chaqirishimiz kerak. Demak, **Employee** konstruktorida **Person** klassi konstruktorini chaqirish zarur.

Buning uchun **super()** ifodasi ishlatiladi. Masalan, **Employee** konstruktorida quyidagi chaqiriq bajariladi:

```
super().__init__(name)
```

Bu ifoda **Person** klassi konstruktorini chaqiradi va unga xodimning ismi uzatiladi. Bu mantiqan toʻgʻri, chunki xodimning ismini **Person** klassi konstruktori oʻrnatadi. **Employee** konstruktorida esa faqat `company` xossasi qoʻshiladi.

Bundan tashqari, **Employee** klassida `display_info()` metodi ham qayta aniqlanadi. Unga kompaniya haqida maʼlumot chiqarish qoʻshiladi. Bu metodni quyidagicha yozish ham mumkin edi:

```
def display_info(self):
    print(f"Name: {self.name}")
    print(f"Company: {self.company}")
```

Ammo bunda ismni chiqarish qismi **Person** klassidagi kodni takrorlaydi. Agar kod bir xil boʻlsa, uni qayta yozishga hojat yoʻq, shuning uchun yana **super()** yordamida **Person** klassidagi `display_info` metodini chaqiramiz:

```
def display_info(self):
    super().display_info()      # Person klassidagi display_info chaqiriladi
    print(f"Company: {self.company}")
```

Shundan soʻng biz **Employee** obyektini yaratib, `display_info()` metodini chaqirishimiz mumkin:

```
tom = Employee("Tom", "Microsoft")
tom.display_info()
```

Konsol chiqishi:

```
Name: Tom
Company: Microsoft
```

Obyekt turini tekshirish

Obyektlar bilan ishlaganda ba'zan ularning turiga qarab turli amallarni bajarish kerak bo'ladi. Buning uchun **isinstance()** funksiyasi qo'llanadi. Bu funksiya ikkita parametr qabul qiladi:

```
isinstance(object, type)
```

- Birinchi parametr — obyekt.
- Ikkinchi parametr — tekshiriladigan klass turi.

Agar obyekt ko'rsatilgan turga mansub bo'lsa, `True`, aks holda `False` qaytaradi.

Masalan, quyidagi **Person** → **Employee/Student** ierarxiasini olaylik:

```
class Person:

    def __init__(self, name):
        self.__name = name    # odamning ismi

    @property
    def name(self):
        return self.__name

    def do_nothing(self):
        print(f"{self.name} does nothing")

# xodim klassi
class Employee(Person):

    def work(self):
        print(f"{self.name} works")

# talaba klassi
class Student(Person):

    def study(self):
        print(f"{self.name} studies")

def act(person):
    if isinstance(person, Student):
        person.study()
    elif isinstance(person, Employee):
        person.work()
    elif isinstance(person, Person):
        person.do_nothing()
```

```
tom = Employee("Tom")
bob = Student("Bob")
sam = Person("Sam")

act(tom)    # Tom works
act(bob)    # Bob studies
act(sam)    # Sam does nothing
```

Bu yerda:

- **Employee** klassida `work()` metodi,
- **Student** klassida `study()` metodi bor.

`act()` funksiyasi **`isinstance()`** orqali obyekt turini tekshiradi va unga mos metodni chaqiradi.

Klass atributlari va statik metodlar

Klass atributlari

Obyekt atributlaridan tashqari, klassda klass atributlari ham aniqlanishi mumkin. Bunday atributlar klass darajasidagi o'zgaruvchilar sifatida belgilanadi. Masalan:

```
class Person:
    type = "Person"
    description = "Describes a person"

print(Person.type)          # Person
print(Person.description)   # Describes a person

Person.type = "Class Person"
print(Person.type)          # Class Person
```

Bu yerda `Person` klassida ikkita atribut aniqlangan:

- `type` — klass nomini saqlaydi,
- `description` — klass ta'rifini saqlaydi.

Klass atributlariga murojaat qilish uchun klass nomidan foydalanamiz, masalan: `Person.type`. Ularni obyekt atributlari kabi o'qish va o'zgartirish mumkin.

Shu bilan birga, ular **barcha obyektlar uchun umumiy** bo'ladi:

```
class Person:
    type = "Person"
    def __init__(self, name):
        self.name = name

tom = Person("Tom")
bob = Person("Bob")
print(tom.type)      # Person
print(bob.type)      # Person

# klass atributini o'zgartiramiz
Person.type = "Class Person"
print(tom.type)      # Class Person
print(bob.type)      # Class Person
```

Klass atributlari barcha obyektlarga tegishli umumiy ma'lumotlarni belgilash uchun ishlatiladi. Masalan, quyidagi kodda `default_name` atributi ism bo'sh kiritilganda obyektga standart qiymat beradi:

```
class Person:
    default_name = "Undefined"
```

```
def __init__(self, name):
    if name:
        self.name = name
    else:
        self.name = Person.default_name

tom = Person("Tom")
bob = Person("")
print(tom.name) # Tom
print(bob.name) # Undefined
```

Klass atributiga metod ichida murojaat qilish uchun klass nomidan foydalaniladi:

```
self.name = Person.default_name
```

Klass atributi va obyekt atributi

Ba'zi hollarda obyekt atributi va klass atributi **bir xil nomga ega** bo'lishi mumkin. Agar obyekt uchun atribut qiymati belgilanmagan bo'lsa, klass atributidan foydalaniladi:

```
class Person:
    name = "Undefined"

    def print_name(self):
        print(self.name)

tom = Person()
bob = Person()
tom.print_name() # Undefined
bob.print_name() # Undefined

bob.name = "Bob"
bob.print_name() # Bob
tom.print_name() # Undefined
```

Bu yerda bob obyektiga yangi name atributi qo'yiladi, ammo tom obyektida klass atributi ishlashda davom etadi.

Agar klass atributini o'zgartirsak, tom uchun ham yangilanadi:

```
tom = Person()
bob = Person()

tom.print_name() # Undefined
bob.print_name() # Undefined

Person.name = "Some Person" # klass atributi o'zgartirildi
bob.name = "Bob" # obyekt atributi belgilandi
bob.print_name() # Bob
```

```
tom.print_name()    # Some Person
```

Statik metodlar

Oddiy metodlardan tashqari klassda **statik metodlar** ham bo'lishi mumkin. Ular `@staticmethod` dekoratori bilan belgilanadi va obyektga bog'liq emas, balki butun klassga tegishlidir.

```
class Person:
    __type = "Person"

    @staticmethod
    def print_type():
        print(Person.__type)

Person.print_type()    # Person - klass nomi orqali chaqirish

tom = Person()
tom.print_type()       # Person - obyekt orqali chaqirish
```

Bu yerda:

- `__type` — klass darajasidagi atribut, u ikki pastki chiziq (`__`) bilan boshlangani uchun **yashirin (private)** hisoblanadi, tashqaridan o'zgartirilmaydi.
- `print_type()` — statik metod bo'lib, u obyektga bog'liq emas va butun klass uchun bir xil qiymatni qaytaradi.

object klassi. Obyektning satr ko‘rinishi

3-versiyadan boshlab Python dasturlash tilida barcha klasslar yashirin tarzda umumiy bitta superclass – `object` ga ega va barcha klasslar sukut bo‘yicha uning metodlarini meros qilib oladi.

`object` klassining eng ko‘p qo‘llaniladigan metodlaridan biri bu `__str__()` metodi. Obyektning satr ko‘rinishini olish yoki obyektning satr ko‘rinishida chiqarish kerak bo‘lganda Python aynan shu metodni chaqiradi. Shuning uchun klassni aniqlashda ushbu metodni qayta aniqlash (override qilish) yaxshi amaliyot hisoblanadi.

Misol uchun, `Person` klassini olaylik va uning satr ko‘rinishini chiqaramiz:

```
class Person:
    def __init__(self, name, age):
        self.name = name # ismni o'rnatamiz
        self.age = age   # yoshni o'rnatamiz

    def display_info(self):
        print(f"Name: {self.name} Age: {self.age}")

tom = Person("Tom", 23)
print(tom)
```

Dastur ishga tushirilganda quyidagiga o‘xshash natija beradi:

```
<__main__.Person object at 0x10a63dc00>
```

Bu obyekt haqida uncha ham ma’lumot bermaydi. Albatta, biz klass ichida qo‘shimcha metod yozib, obyekt ma’lumotlarini chiqarishimiz mumkin – yuqoridagi misolda bu `display_info` metodi.

Lekin bundan ham qulay yo‘l bor – `Person` klassida `__str__()` metodini aniqlash:

```
class Person:
    def __init__(self, name, age):
        self.name = name # ismni o'rnatamiz
        self.age = age   # yoshni o'rnatamiz

    def display_info(self):
        print(self)
        # yoki shunday ham yozish mumkin:
        # print(self.__str__())

    def __str__(self):
        return f"Name: {self.name} Age: {self.age}"

tom = Person("Tom", 23)
print(tom)           # Name: Tom Age: 23
tom.display_info()   # Name: Tom Age: 23
```


`__str__` metodi satrni qaytarishi kerak. Bu misolda biz odam haqidagi asosiy ma'lumotlarni qaytardik. Agar shu ma'lumotni boshqa metodlarda ishlatmoqchi bo'lsak, `self.__str__()` ifodasidan foydalanishimiz mumkin.

Endi konsolda quyidagicha chiqadi:

```
Name: Tom Age: 23
Name: Tom Age: 23
```

Operatorlarni ortiqcha yuklash

Python o'z klasslari uchun qo'shish, ayirish va boshqa standart operatorlarni belgilash imkonini beradi. Buning uchun `operator` modulida bir qator funksiyalar mavjud:

Amallar	Sintaksis	Funksiya
Qo'shish	$a + b$	<code>__add__(a, b)</code>
Birlashtirish	$seq1 + seq2$	<code>__concat__(seq1, seq2)</code>
Mavjudligini tekshirish	$obj \text{ in } seq$	<code>__contains__(seq, obj)</code>
Bo'lish	a / b	<code>__truediv__(a, b)</code>
Bo'lish (butun)	$a // b$	<code>__floordiv__(a, b)</code>
Bitwise AND	$a \& b$	<code>__and__(a, b)</code>
Bitwise XOR	$a \wedge b$	<code>__xor__(a, b)</code>
Bitwise inversiya	$\sim a$	<code>__invert__(a)</code>
Bitwise OR	$a b$	<code>__or__(a, b)</code>
Daraja	$a ** b$	<code>__pow__(a, b)</code>
Indeks bo'yicha tayinlash	$obj[k] = v$	<code>__setitem__(obj, k, v)</code>
Indeks bo'yicha o'chirish	$del\ obj[k]$	<code>__delitem__(obj, k)</code>
Indeks bo'yicha murojaat	$obj[k]$	<code>__getitem__(obj, k)</code>
Chapga siljitish	$a \ll b$	<code>__lshift__(a, b)</code>
Qoldiq	$a \% b$	<code>__mod__(a, b)</code>
Ko'paytirish	$a * b$	<code>__mul__(a, b)</code>
Matritsa ko'paytmasi	$a @ b$	<code>__matmul__(a, b)</code>
Manfiy qiymat	$-a$	<code>__neg__(a)</code>
Mantiqiy inkor	$not\ a$	<code>__not__(a)</code>
Musbat qiymat	$+a$	<code>__pos__(a)</code>
O'ngga siljitish	$a \gg b$	<code>__rshift__(a, b)</code>
Oraliqqa tayinlash	$seq[i:j] = values$	<code>__setitem__(seq, slice(i, j), values)</code>
Oraliqni o'chirish	$del\ seq[i:j]$	<code>__delitem__(seq, slice(i, j))</code>
Oraliqqa murojaat	$seq[i:j]$	<code>__getitem__(seq, slice(i, j))</code>
Ayirish	$a - b$	<code>__sub__(a, b)</code>
True/False tekshirish	obj	<code>__bool__(obj)</code>
Kichik	$a < b$	<code>__lt__(a, b)</code>
Kichik yoki teng	$a \leq b$	<code>__le__(a, b)</code>
Teng	$a == b$	<code>__eq__(a, b)</code>
Teng emas	$a \neq b$	<code>__ne__(a, b)</code>
Katta yoki teng	$a \geq b$	<code>__ge__(a, b)</code>
Katta	$a > b$	<code>__gt__(a, b)</code>
Qo'shish va tayinlash	$a += b$	<code>__iadd__(a, b)</code>
Birlashtirish va tayinlash	$a += b$	<code>__iconcat__(a, b)</code>
Bitwise AND va tayinlash	$a \&= b$	<code>__iand__(a, b)</code>

Amallar	Sintaksis	Funksiya
Butun bo'lish va tayinlash	<code>a //= b</code>	<code>__floordiv__(a, b)</code>
Chapga siljitish va tayinlash	<code>a <<= b</code>	<code>__lshift__(a, b)</code>
O'ngga siljitish va tayinlash	<code>a >>= b</code>	<code>__rshift__(a, b)</code>
Qoldiq bo'yicha bo'lish va tayinlash	<code>a %= b</code>	<code>__imod__(a, b)</code>
Ko'paytirish va tayinlash	<code>a *= b</code>	<code>__imul__(a, b)</code>
Matritsa ko'paytmasi va tayinlash	<code>a @= b</code>	<code>__imatmul__(a, b)</code>
Bitwise OR va tayinlash	<code>a = b</code>	<code>__ior__(a, b)</code>
Daraja va tayinlash	<code>a **= b</code>	<code>__ipow__(a, b)</code>
Ayirish va tayinlash	<code>a -= b</code>	<code>__isub__(a, b)</code>
Bo'lish va tayinlash	<code>a /= b</code>	<code>__itruediv__(a, b)</code>
XOR va tayinlash	<code>a ^= b</code>	<code>__ixor__(a, b)</code>

Ma'lum bir klass uchun operatorni belgilash uchun, ushbu klass mos funksiyani amalga oshirishi kerak. Masalan, qo'shish operatorini belgilash uchun `__add__()` funksiyasidan foydalaniladi, shuning uchun klass ichida bu funksiyani aniqlash kerak.

Misol:

```
class Counter:
    def __init__(self, value):
        self.value = value
    # qo'shish operatorini belgilash
    def __add__(self, other):
        return Counter(self.value + other.value)

counter1 = Counter(5)
counter2 = Counter(15)
counter3 = counter1 + counter2
print(counter3.value)  # 20
```

Bu yerda `Counter` klassi mavjud bo'lib, u `value` atributiga ega — bu shartli raqam. `__add__` funksiyasi yordamida `Counter` turiga qo'shish operatori berildi. Agar biz xohlasak, bitta `Counter` obyekti boshqa `Counter` obyekti bilan qo'shilsin, unda funksiyaning ikkinchi parametri boshqa `Counter` obyekti bo'ladi:

```
def __add__(self, other):
    return Counter(self.value + other.value)
```

Natijada yangi `Counter` obyekti qaytariladi, unga ikkala obyektning `value` atributlari yig'iladi. Shu tarzda ikkita `Counter` obyektni qo'shish mumkin bo'ladi va natija yangi `Counter` obyekti bo'ladi.

Ammo bu qo'shish operatorining yagona varianti emas. Masalan, agar biz `Counter` obyektni boshqa `Counter` bilan emas, balki oddiy son bilan qo'shmoqchi bo'lsak, operatorni quyidagicha belgilash mumkin:

```
class Counter:
```

```

def __init__(self, value):
    self.value = value

def __add__(self, other):
    return Counter(self.value + other)

counter1 = Counter(5)
counter3 = counter1 + 6
print(counter3.value) # 11

```

Bu yerda qo‘shish operatori hali ham `Counter` obyektini qaytaradi, faqat ikkinchi parametrd oddiy sonni ifodalaydi.

Qaytariladigan tur bir qator operatorlar uchun ham qat’iy belgilanmagan. Masalan, biz oddiy sonni ham qaytarishimiz mumkin:

```

class Counter:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other

counter1 = Counter(5)
result = counter1 + 7
print(result) # 12

```

Operatorlarni belgilashning boshqa misollari:

Obyektning haqiqat qiymati

`__bool__()` funksiyasini belgilash obyektning True/False qiymatini aniqlash imkonini beradi. Masalan:

```

class Counter:
    def __init__(self, value):
        self.value = value
    def __bool__(self):
        return self.value > 0

def test(counter):
    if counter:
        print("Counter = True")
    else:
        print("Counter = False")

counter1 = Counter(3)
test(counter1) # Counter = True

counter2 = Counter(-3)
test(counter2) # Counter = False

```

Bu yerda `value` qiymati 1 dan kichik bo'lsa, `Counter` obyekt `False`, musbat qiymatda esa `True` hisoblanadi. Shu sababli `Counter` obyektini shartli yoki sikl konstruktsiyalarida ishlatish mumkin.

Masalan, `while` siklida:

```
counter1 = Counter(3)

while(counter1):
    print("Counter1: ", counter1.value)
    counter1.value -= 1
```

Sikl `counter1.value > 0` ekanida davom etadi.

Bool qiymat qaytaradigan operatorlar

Ba'zi operatorlar logik `True/False` qiymat qaytaradi, masalan solishtirish operatorlari:

```
class Counter:
    def __init__(self, value):
        self.value = value

    def __gt__(self, other):
        return self.value > other.value
    def __lt__(self, other):
        return self.value < other.value

counter1 = Counter(1)
counter2 = Counter(2)

if counter1 > counter2:
    print("counter1 counter2 dan katta")
elif counter1 < counter2:
    print("counter1 counter2 dan kichik")
else:
    print("counter1 va counter2 teng")
```

Bu yerda `< (__lt__())` va `> (__gt__())` operatorlari boshqa `Counter` obyektlari bilan solishtirish uchun ishlatiladi. Amalda esa ikkita obyektning `value` atributlarini solishtiramiz:

```
def __gt__(self, other):
    return self.value > other.value
def __lt__(self, other):
    return self.value < other.value
```

Shundan so'ng ikkita `Counter` obyektiga solishtirish operatorlarini qo'llash mumkin:

```
if counter1 > counter2:
```

Indeks orqali murojaat qilish operatorlari

Ba'zi operatorlar obyektga indeks orqali murojaat qilish imkonini beradi, kvadrat qavslar yordamida:

```
obj[index]
```

Odatiy holatda bunday operatsiyalar kolleksiyalarga nisbatan qoʻllanadi, lekin amalda har qanday obyektga qoʻllanishi mumkin, indeks esa istalgan qiymat boʻlishi mumkin. Masalan, indeks boʻyicha qiymat olishni koʻrib chiqamiz:

```
class Person:
    def __init__(self, name, age):
        self.__name = name
        self.__age = age

    def __getitem__(self, prop):
        if prop == "name": return self.__name
        elif prop == "age": return self.__age
        return None

tom = Person("Tom", 39)
print("Name:", tom["name"]) # Name: Tom
print("Age:", tom["age"])   # Age: 39
print("Id:", tom["id"])     # Id: None
```

Bu yerda `Person` klassida ikkita privet maydon `__name` va `__age` mavjud. `__getitem__()` funksiyasi indeks boʻyicha qiymat olishni taʼminlaydi. Indeks atribut nomi sifatida ishlatiladi va mos ravishda `__name` yoki `__age` qiymati qaytariladi. Notoʻgʻri atribut nomi berilsa, `None` qaytariladi.

```
tom["name"]
```

Xususiyat mavjudligini tekshirish

`in` operatori maʼlum qiymatning ketma-ketlikda mavjudligini tekshiradi:

```
value in sequence
```

Agar qiymat ketma-ketlikda boʻlsa, `True`, aks holda `False` qaytariladi. Misol:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __contains__(self, prop):
        if prop == "name" or prop == "age": return True
        return False

tom = Person("Tom", 39)
print("name in tom") # True
print("id in tom")   # False
```

`in` operatorining ishlashini `__contains__()` funksiyasi ta'minlaydi. Birinchi parametr - obyekt, ikkinchi parametr - tekshirilayotgan qiymat. Agar qiymat `"name"` yoki `"age"` bo'lsa, `True` qaytariladi. Shu sababli:

```
"name" in tom # True
"id" in tom   # False
```

Operatorlarni juft holda amalga oshirish

Ba'zi operatorlar, masalan solishtirish operatorlari, juft holda qulayroq ishlatiladi. Agar `==` operatori belgilansa, `!=` operatorini ham birga belgilash mumkin. Shu bilan birga bir xil logikani ikki marta yozmaslik uchun bir operatorni boshqasidan hosil qilish mumkin:

```
class Counter:
    def __init__(self, value):
        self.value = value

    def __eq__(self, other): return self.value == other.value
    def __ne__(self, other): return not (self == other)

    def __gt__(self, other): return self.value > other.value
    def __le__(self, other): return not (self > other)

    def __lt__(self, other): return self.value < other.value
    def __ge__(self, other): return not (self < other)

c1 = Counter(1)
c2 = Counter(2)

print(c1 == c2) # False
print(c1 != c2) # True
print(c1 < c2)  # True
print(c1 >= c2) # False
```

Bu yerda `!=` operatori `==` operatorining teskari natijasini qaytaradi, `<` va `>=`, `>` va `<=` operatorlari ham shunday juft sifatida belgilangan.

Abstrakt klasslar va metodlar

Odatda klasslar atrof-muhitdagi ayrim obyektlarni aks ettiradi. Ammo baʼzan biz konkret tashqi koʻrinishga ega boʻlmagan mavjudotlar bilan ishlashimiz kerak boʻladi. Masalan, “hayvon” mavjudoti. Aniq hayvonlar bor — mushuk, it va hokazo, lekin hayvon oʻzi konkret koʻrinishga ega emas. Yoki “geometrik figura” mavjudoti. Aniq figuralar bor — toʻgʻri toʻrtburchak, kvadrat, doira, uchburchak, lekin geometrik figura oʻzi konkret koʻrinishga ega emas. Shunday mavjudotlarni tasvirlash uchun odatda abstrakt klasslar ishlatiladi.

Python tilida abstrakt klasslar yaratish uchun barcha vositalar **abc** modulida aniqlangan va bu modul dasturga qoʻshimcha tarzda import qilinishi kerak:

```
import abc
```

Ushbu modulning asosiy komponentlari — **ABC** klassi va **@abstractmethod** annotatsiyasi. ABC klassi abstrakt klass yaratishni osonlashtiradi, barcha abstrakt klasslar ushbu klassdan meros oladi. **@abstractmethod** annotatsiyasi esa abstrakt metod yaratish uchun moʻljallangan.

Abstrakt klasslar odatiy klasslar kabi aniqlanadi, faqat ular **abc** modulidagi ABC klassidan meros oladi. Masalan, geometrik figura uchun abstrakt klass quyidagicha aniqlanishi mumkin:

```
import abc
class Shape(abc.ABC):
    pass
```

Odatda abstrakt klasslar merosxoʻr klasslar uchun umumiy funksionallikni eʼlon qiladi. Shu bilan birga, baʼzi funksionalliklar hech qanday realizatsiyaga ega boʻlmasligi mumkin — ularni merosxoʻr klasslar aniqlaydi. Bunday funksionallik abstrakt metod sifatida aniqlanadi. Masalan, geometrik figura klassida perimetr va yuzani hisoblash metodlari boʻlishi mumkin. Barcha figuralar uchun umumiy formula yoʻqligi sababli, **area** metodini abstrakt sifatida belgilaymiz:

```
import abc
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self):
        pass          # figuraning maydoni
```

Bu holatda **area()** metodi abstrakt sifatida aniqlangan va unga konkret funksionallik kerak emas, shuning uchun **pass** operatori ishlatiladi.

Shuni taʼkidlash kerakki, biz abstrakt metodlarga ega boʻlgan abstrakt klassning obyektini bevosita uning konstruktori orqali yaratolmaymiz:

```
import abc
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self): pass          # figuraning maydoni

shape = Shape()          # ! Xato - bunday qilish mumkin emas
print(shape)
```


Merosxoʻr klasslar abstrakt klassdagi barcha abstrakt metodlarni amalga oshirishi kerak. Masalan, toʻgʻri toʻrtburchak klassini aniqlaymiz:

```
import abc
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self): pass          # figuraning maydoni

# Toʻgʻri toʻrtburchak klassi
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self): return self.width * self.height

rect = Rectangle(30, 50)
print("Rectangle area:", rect.area())    # Rectangle area: 1500
```

Bu yerda `Rectangle` klassi konstruktori orqali kenglik va balandlikni qabul qiladi va ularni `area()` metodida maydonni hisoblash uchun ishlatadi.

Shuningdek, boshqa turdagi figuralar ham aniqlanishi mumkin. Masalan, doira klassini qoʻshamiz:

```
import abc
class Shape(abc.ABC):
    @abc.abstractmethod
    def area(self): pass          # figuraning maydoni

# Toʻgʻri toʻrtburchak klassi
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self): return self.width * self.height

# Doira klassi
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius
    def area(self): return self.radius * self.radius * 3.14

def print_area(shape):
    print("Area:", shape.area())

rect = Rectangle(30, 50)
circle = Circle(30)
print_area(rect)          # Area: 1500
print_area(circle)        # Area: 2826.0
```

Bu holatda `print_area` funksiyasi har qanday figura obyektini qabul qiladi va uning maydonini chiqaradi.

Shuningdek, abstrakt klasslar konstruktori, atributlar va abstrakt bo'lmagan metodlarni ham aniqlay oladi, ular merosxo'r klasslarda ham ishlatilishi mumkin:

```
import abc
class Shape(abc.ABC):
    def __init__(self, x, y):
        self.x = x
        self.y = y

    @abc.abstractmethod
    def area(self): pass          # abstrakt metod

    def print_point(self):        # abstrakt bo'lmagan metod
        print("X:", self.x, "\tY:", self.y)

# To'g'ri to'rtburchak klassi
class Rectangle(Shape):
    def __init__(self, x, y, width, height):
        super().__init__(x, y)
        self.width = width
        self.height = height
    def area(self): return self.width * self.height

rect = Rectangle(10, 20, 100, 100)
rect.print_point()              # X: 10   Y: 20
```

Bu yerda `Shape` abstrakt klassi konstruktori orqali `X` va `Y` koordinatalarini qabul qiladi (masalan, to'g'ri to'rtburchak uchun yuqori chap burchak koordinatalari, doira uchun markaz koordinatalari). Shuningdek, `print_point` abstrakt bo'lmagan metod sifatida koordinatalarni konsolga chiqaradi.